Problem Set 3

Sagar Lee Stockwell Clemensen (FCK916)

Advanced Methods in Applied Statistics

11th of March 2026

Problem 1

Problem 1a

I chose the first 500 entries of high earners and low earners as test data. The rest I used for training. In the training data there were 3139 low earners and 661 high earners. Using a threshold of -0.414 for the Boosted Decision Tree score (BDT score) to distinguish between high earners (BDT score > -0.414) and low earners (BDT score ≤ -0.414), I achieved an accuracy of 0.768 on the testing data.

The histogram showing the 500 high and low earners from the testing data as a function of BDT score is shown in Fig. 1. The threshold for distinguishing is also plotted. I have picked low-income earners as the background and high-income earners as the signal.

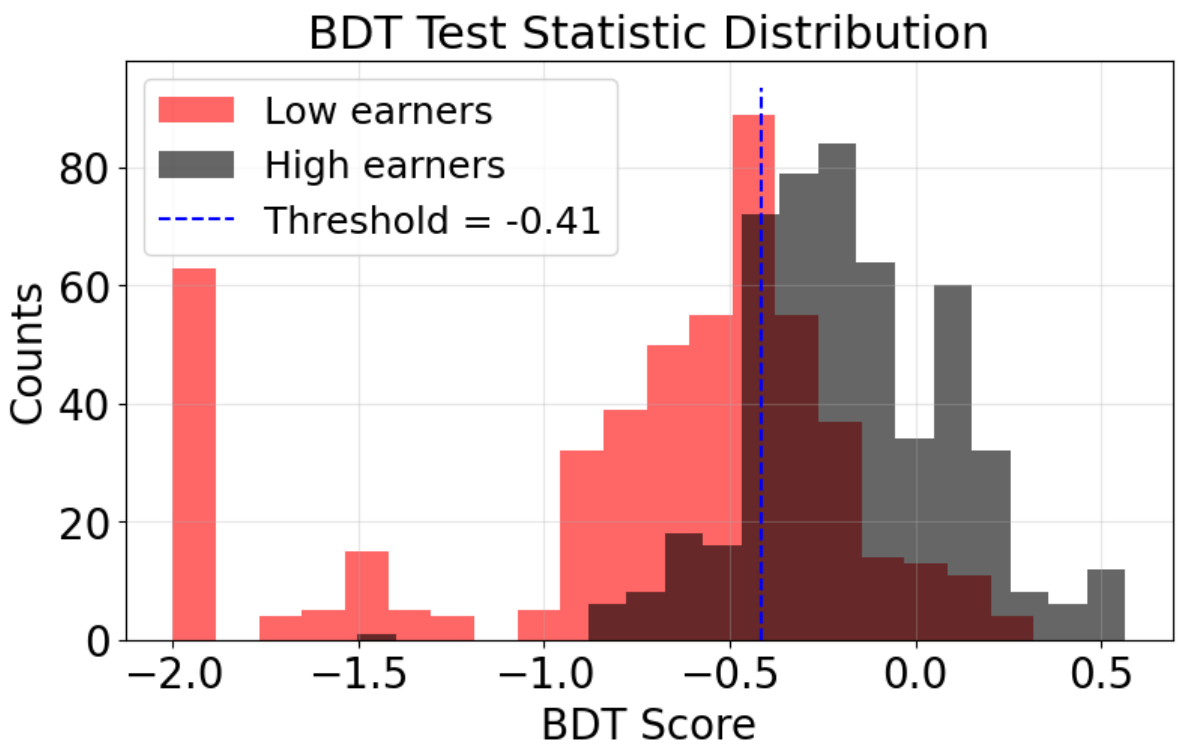


Figure 1: Histogram showing the 500 high and low earners from the testing data as a function of BDT score.

Problem 1b

Low earners	High earners
0.826	0.174

Table 1: Fraction of low earners and high earners in the training data.

Overtraining occurs when a machine learning model learns statistical fluctuations in the training data instead of the underlying patterns. This typically leads to very good performance on the training data but poorer performance on unseen data. To reduce the risk of overtraining, I limited the complexity of the Boosted Decision Tree

using shallow decision trees as base learners and a moderate learning rate. This prevents the model from becoming overly sensitive to individual training samples. To ensure that the classifier does not exhibit signs of overtraining I performed some checks. The classifier output distributions for the testing data shown in Fig. 2 show a clear but not perfect separation between the two classes, indicating that the model is learning general features rather than memorizing the training data. In addition, I performed a 5-fold cross-validation. This gave an average accuracy of 0.802 with a standard deviation of 0.009. The small variation between folds indicates that the model performance is stable and does not depend strongly on a particular data split, suggesting that the classifier generalizes well.

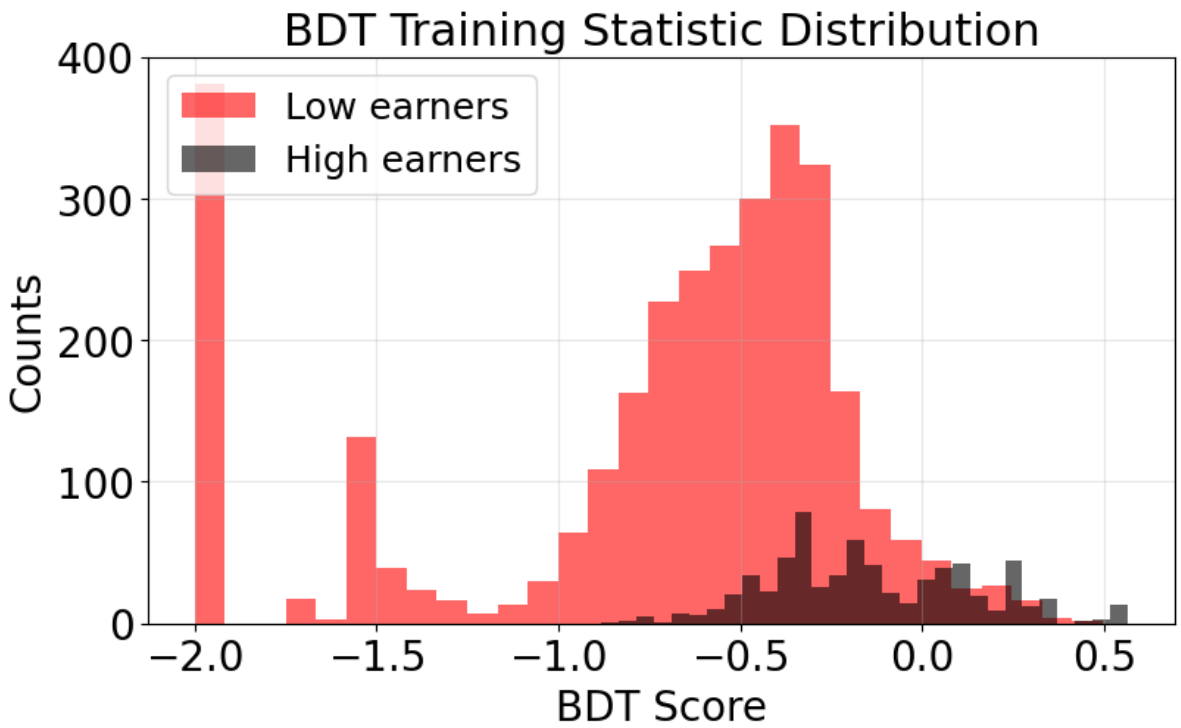


Figure 2: Histogram of decision scores for the training data.

Problem 1c

The files are labeled "sagar_clemensen.low_ID.txt" and "sagar_clemensen.high_ID.txt".

Problem 2

Problem 2a

If a car crash entry contained values for both `lon` and `lon2` (and similarly for `lat` and `lat2`), the value in `lon` and `lat` was used. If the entry in `lon` or `lat` was missing, the corresponding value in `lon2` or `lat2` was used instead.

Some entries in the data file contained no values for any of the columns `lon`, `lat`, `lon2`, or `lat2`. These observations could therefore not be included in the analysis.

Entries that contained only a longitude value (`lon` or `lon2`) or only a latitude value (`lat` or `lat2`) were also excluded, since both coordinates are required to determine a location.

In addition, entries corresponding to locations far outside Cary were removed. Car crashes were considered to occur in Cary if the latitude lay in the interval $[33.781944, 37.781944]$ and the longitude lay in the interval $[-80.82, -76.82]$[1].

However, not all observations outside this interval were excluded. Some entries appeared to contain incorrectly placed decimal separators. For example, the measurement "Lat: 35.7385, Lon: −788162.0" appears likely to correspond to "Lat: 35.7385, Lon: −78.81620", where the decimal point was misplaced. Entries of this type were corrected and retained in the dataset.

Besides the entries with no values for either latitude or longitude I excluded 90 entries. These are listed in in the file "removed_indices_log.txt" which is in the zipped file.

The scatter plot of crash entries as a function of the latitude and longitude is shown in Fig. 3.

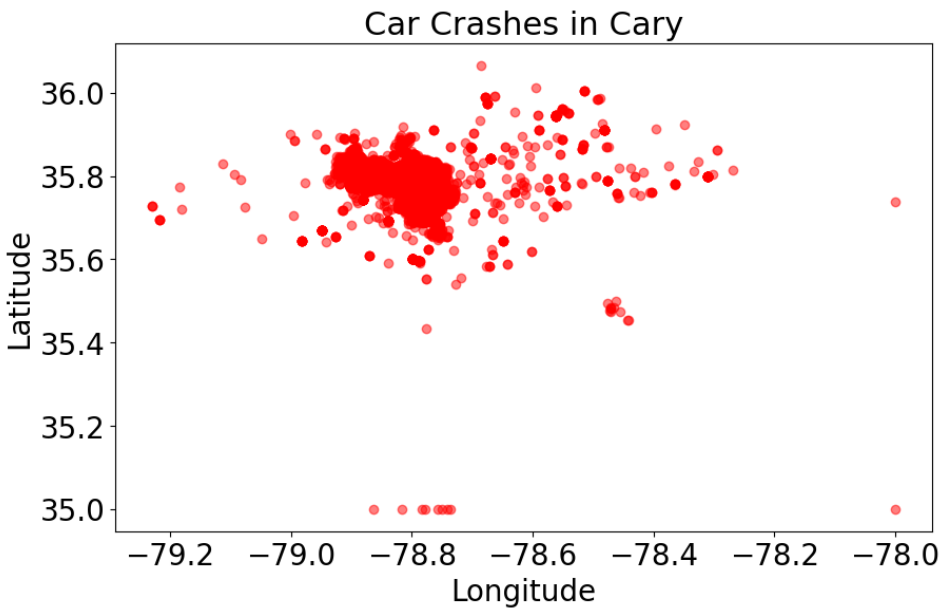


Figure 3: Scatter plot of crash entries in Cary as a function of the latitude and longitude.

A histogram of the time of day of each crash is shown in Fig. 4.

To create a kernel density estimation (KDE) using a gaussian kernel I would start by taking the crash times given in HH:MM format and convert them into decimal hours between 0 and 24. Each crash time is then treated as an observation where a Gaussian kernel is centered at that time. The bandwidth of the kernel, which for a Gaussian is σ, is set to 0.25 hours. This determines the width of each Gaussian curve and therefore how much influence each observation has on nearby times. The KDE is constructed by evaluating the sum of the Gaussian kernels across the range of crash times between 0 and 24 hours. At each point in time, the density value is obtained by summing the contributions from all kernels centered at the observed crash times.

To ensure that the KDE wraps around at 00:00 and 24:00, the time variable must be treated as circular. I would achieve this by extending the data by adding shifted copies of the observations. I would add a copy of the whole data set shifted -24 hours as well as a copy shifted by +24 hours, so that it is equally extended to both sides.

[1]Wikipedia lists the coordinates of Cary as 35.781944, −78.82 Wikipedia. The interval was constructed by extending 2 degrees on either side of the latitude and longitude. Outside of this interval you are no longer in North Carolina.

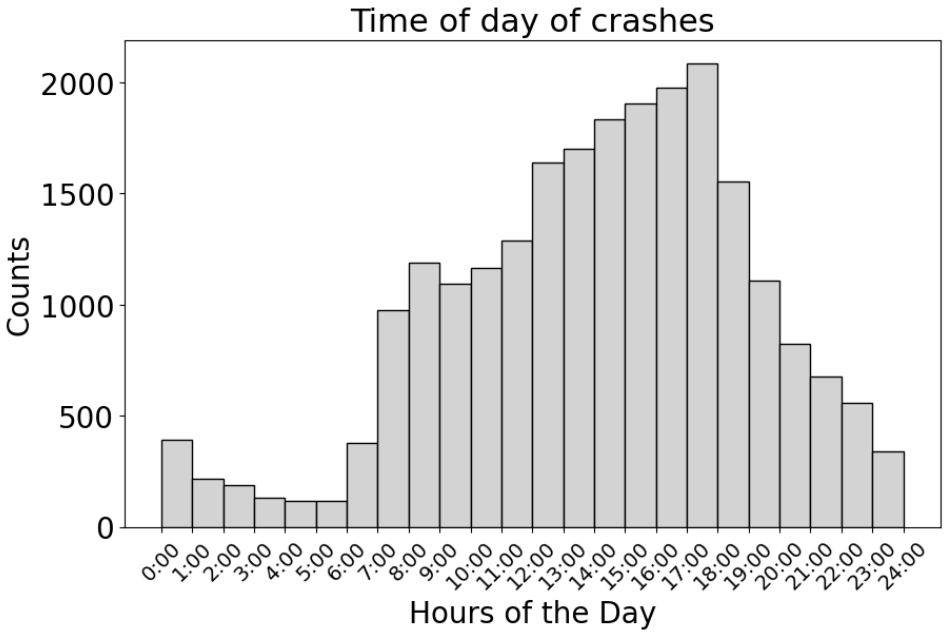


Figure 4: A histogram of the time of day of each crash in Cary.

Then I would compute the KDE over this extended range. I would then restrict the final KDE to only be in the interval $[0, 24]$ and normalize it so the total area in this interval is one. This ensures that observations close to midnight correctly influence the density on both sides of the boundary and that it is still a PDF.

Problem 2b

The plot of the PDF constructed by an Epanechnikov kernel KDE with a bandwidth of 0.8 alongside the histogram from problem 2a is shown in Fig. 5.

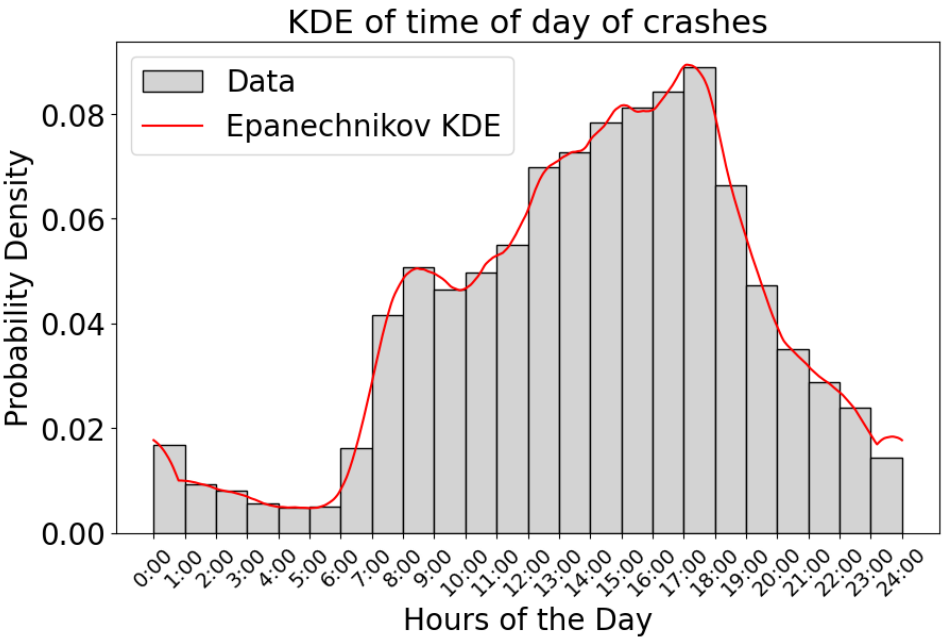


Figure 5: PDF constructed by an Epanechnikov kernel KDE with a bandwidth of 0.8 alongside a histogram of the time of day of each crash in Cary.

x	$f(x)$
00:23	0.0132
01:49	0.0089
08:11	0.0498
15:54	0.0806
18:02	0.0790
21:11	0.0307
23:44	0.0101

Table 2: The evaluation of the PDF f() constructed by the Epanechnikov kernel KDE.

The best two-hour window for the additional police officers to patrol is from 15:58 to 17:58. The 24-hour percentage of crashes would be reduced by 1.72% for this choice of window for patrolling.

Problem 2c

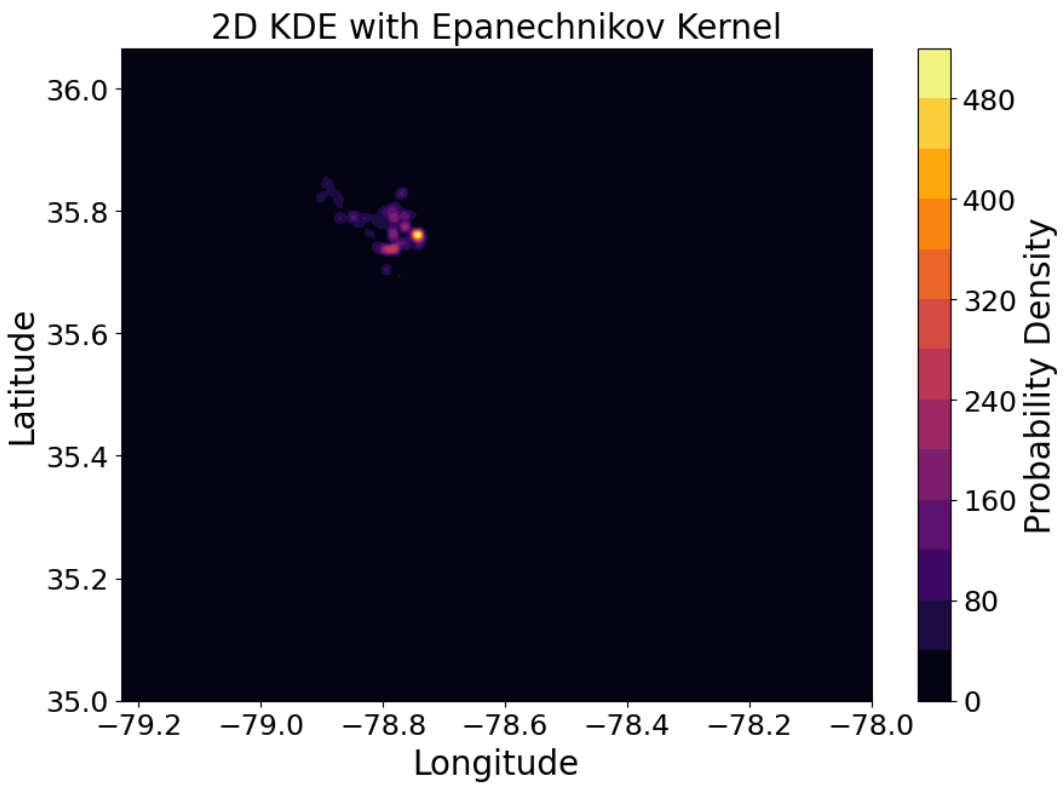


Figure 6: A two-dimensional KDE PDF from the crash data using the latitude and longitude with an Epanechnikov kernel and a bandwidth of 0.01 in both dimensions. Since the probability density is mostly 0 a zoomed-in version is provided Fig. **??**.

Using the KDE PDF seen in Fig. 6 the total percentage of crashes estimated to be within the 'box' of longitude range of $[-78.76, -78.72]$ and latitude range of $[35.74, 35.78]$ is 18.25%.

Fig. 16 in Appendix A shows the estimated PDF overlaid on a screenshot from OpenStreetMap of the corresponding geographic area.

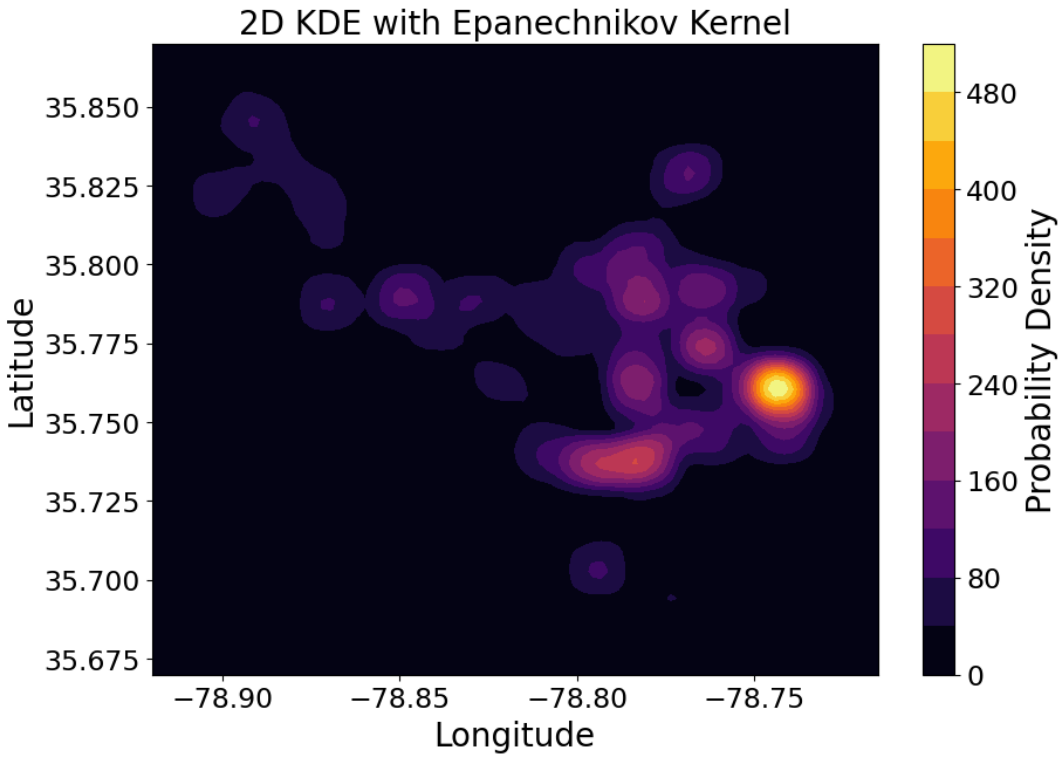


Figure 7: Zoomed-in version of Fig. 6.

Problem 3

Problem 3a

Using Bayes' theorem the following table of probabilities is produced

$P(A_1 \mid D)$	$P(A_2 \mid D)$	$P(A_3 \mid D)$	$P(A_4 \mid D)$	$P(A_5 \mid D)$
21.37%	18.32%	15.27%	21.37%	23.66%

Table 3: Probabilities of a defective pace maker coming from a given facility A_i with $i \in [1, 5]$.

Thus the probability that a defective pace-maker came from the A_2 facility is 18.32%. If a defective pace-maker is found, it is most likely to be from facility A_5.

Problem 3b

If p_i is the production share of facility i and d_i its defect rate, then the probability that a defective pacemaker came from facility i is

$$P(A_i \mid D) = \frac{p_i d_i}{\sum_{j=1}^{5} p_j d_j}. \tag{1}$$

We want all facilities to have the same chance of being identified when a pace-maker fails. Since there are five facilities, this probability should be

$$P(A_i \mid D) = \frac{1}{5}. \tag{2}$$

Setting these equal and letting d_i' be the new defective rate of facility i yields

$$\frac{p_i d_i'}{\sum_{j=1}^{5} p_j d_j'} = \frac{1}{5}. \tag{3}$$

Rewriting this gives the following equation

$$5 p_i d_i' = \sum_{j=1}^{5} p_j d_j' = S. \tag{4}$$

This means the quantity $p_i d_i'$ must be the same for every facility.

Because the defect rates are only allowed to increase, we choose the constant based on the largest current value:

$$S = \max(5 p_i d_i). \tag{5}$$

The updated defect rate for facility i is then

$$d_i' = \frac{S}{5 p_i}. \tag{6}$$

The updated defective rates are shown in Table 4.

Facility	A_1	A_2	A_3	A_4	A_5
Updated defective rate	2.21%	5.17%	15.50%	3.88%	3.10%

Table 4: The updated defective rates for each facility that ensures that all the facilities have the same probability of being identified with a failed pace-maker.

Problem 4

Problem 4a

I used Markov Chain Monte Carlo (MCMC) to sample the probability distribution of the nucleosome position in the potential landscape. I used a discrete system of positions, as the potentials are given as a discrete set. The convergence of the MCMC method is shown in Fig. 8.

In general a Markov Chain has to be ergodic ie. all states can eventually be reached. For the nucleosome sampling when the inverse temperature $\beta = 1$, the sampling becomes strongly dependent on the initial position. Under these conditions, the system seems to lose ergodicity, although this effect may also arise from the discrete nature of the simulation. In any case, a larger inverse temperature corresponds to a lower physical temperature. As a result, the nucleosome tends to remain in the nearest potential minimum and rarely escapes it, since it typically does not receive sufficient thermal energy to overcome the barrier. Nevertheless, even though the Monte Carlo model may no longer be strictly valid due to this apparent loss of ergodicity, the analytical estimate still shows good agreement with the results, as illustrated in Fig. 9.

To visualize the time steps I have plotted the median and 68.27% confidence interval for the amount of steps used to move from every position for the three values of β. The plots are shown in Fig, 10.

The behavior is again explained by the inverse temperature β. At high temperatures (left), the median time to take a step is 1 at all positions, because the temperature is high enough that moving occurs almost every step. At very low temperatures (right) the nucleosome can be stuck at a position for thousands of steps.

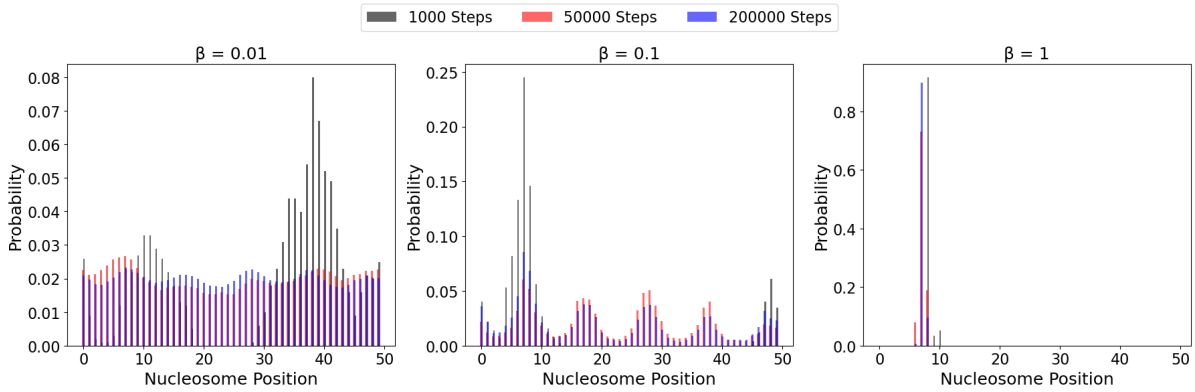


Figure 8: Distribution and convergence of my MCMC sampling.

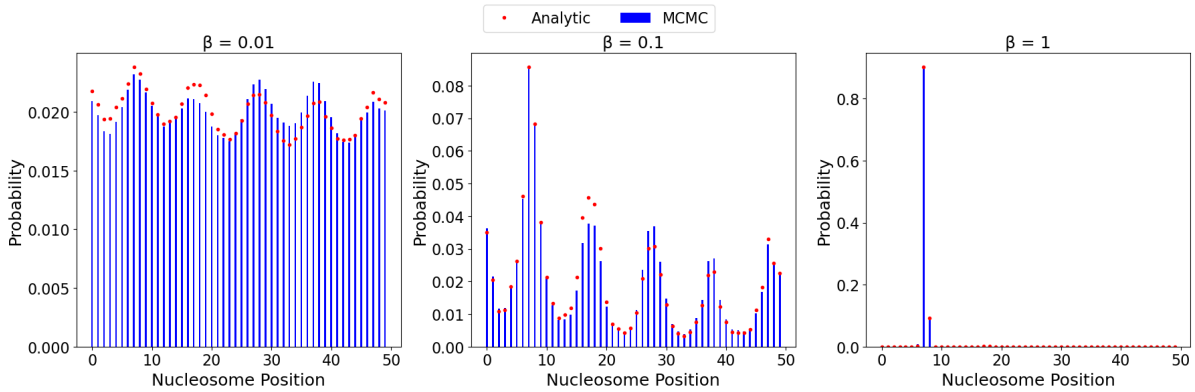


Figure 9: Final distribution of my MCMC sampling alongside the analytic estimate.

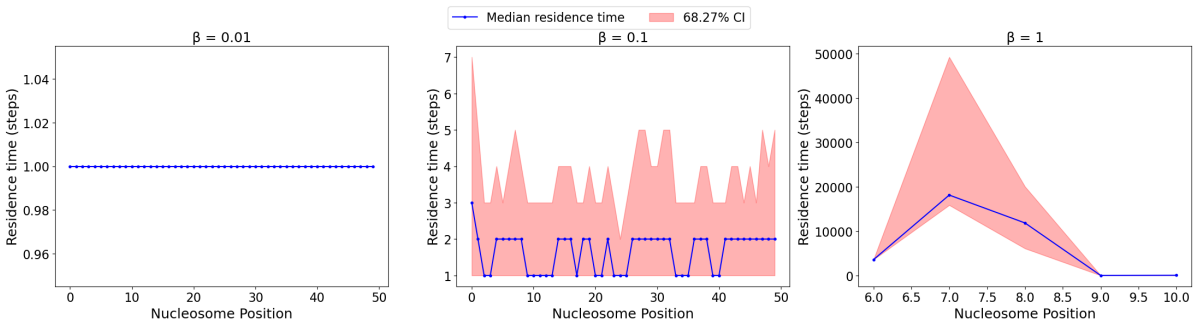


Figure 10: Time step distributions for each position of the nucleosome. With median and 68.27% confidence interval.

Problem 4b

I have implemented the remodeller with $\tau = 25$. The new distribution, with intermediate snapshots for convergence, is shown in Fig. 11.

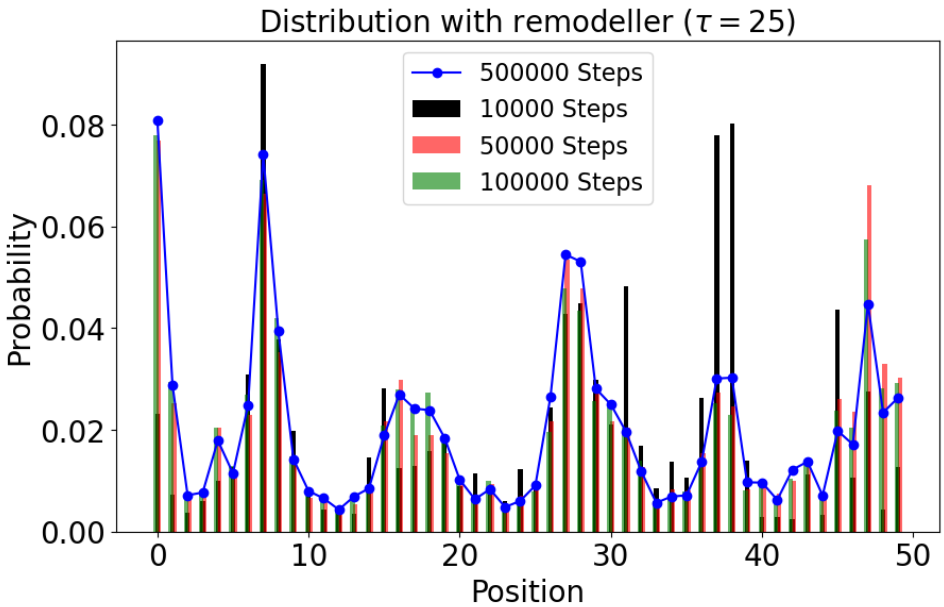


Figure 11: Distribution and convergence of the nucleosome model with a remodeller.

I also sampled for $\tau = 10$ and $\tau = 100$ and compared it to the result for $\tau = 25$. The comparison of the three are shown in Fig. 12. The convergence of the two new final distributions are shown in Fig. 13.

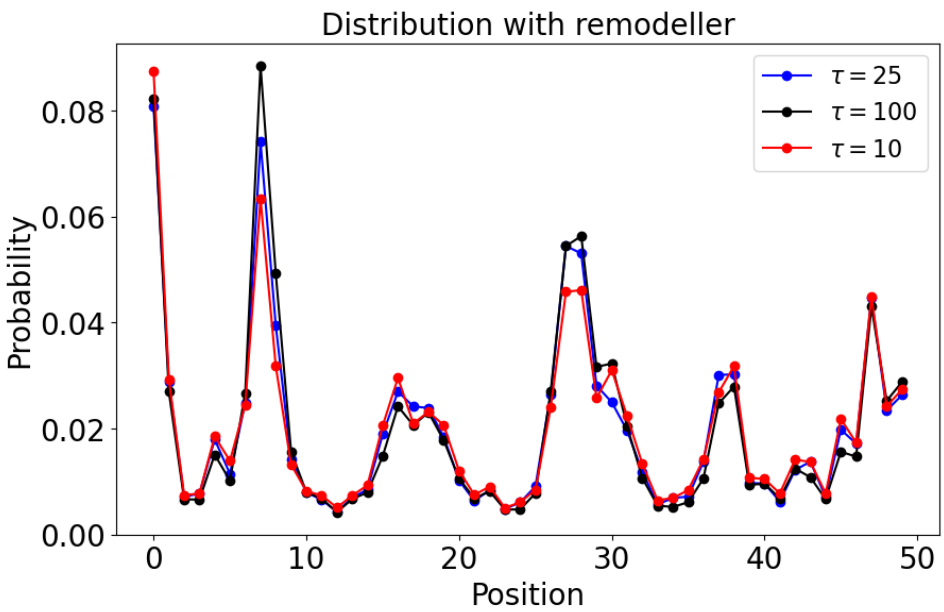
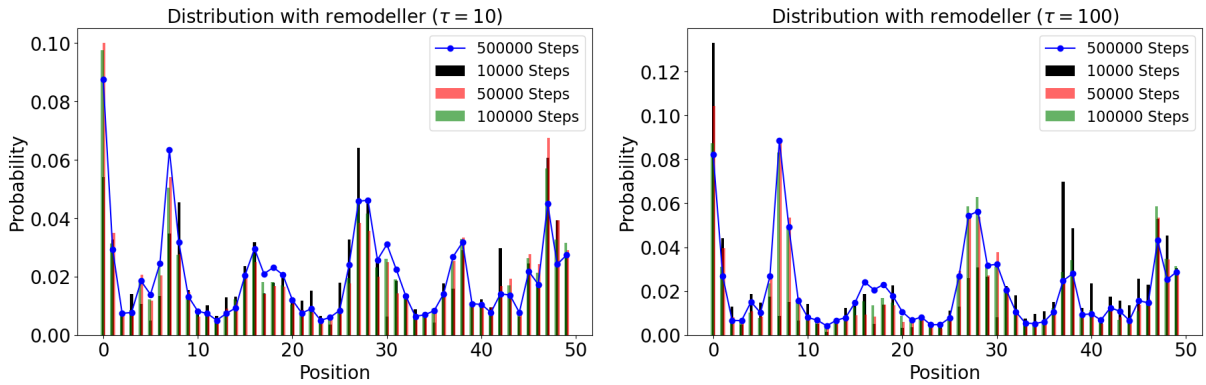


Figure 12: Comparison between final distributions for different rates of τ. Despite positions being discrete, I have decided to draw lines between points to make it easier to visually compare.



(a) Distribution and convergence of the nucleosome model with a remodeller for $\tau = 10$.

(b) Distribution and convergence of the nucleosome model with a remodeller for $\tau = 100$.

Figure 13: Distribution and convergence of the nucleosome model with changed rates τ.

Finally I sampled with the remodeller being biased either to the right (positive x-direction) or left (negative x-direction) and compared them to the unbiased remodeller. The comparison of the three are shown in Fig. 14. As seen in the plot there is definitely a difference between the unbiased and biased distribution. The two unbiased distributions also differ from each other. The most probable positions are roughly the same for all three distributions (same peaks), but the probability value differs between them.

The convergence of the two new final distributions are shown in Fig. 15.

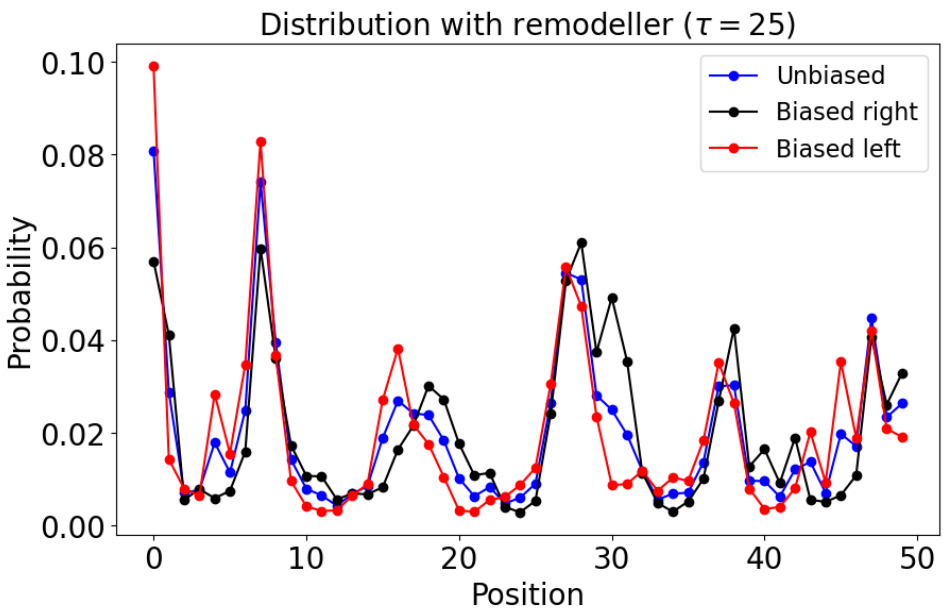
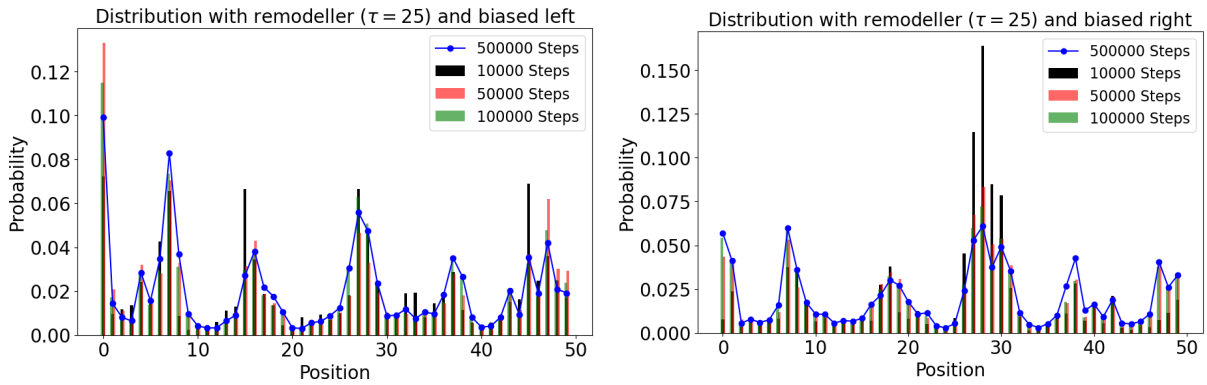


Figure 14: Comparison between final distributions for whether the remodeller is biased or unbiased. Despite positions being discrete, I have again decided to draw lines between points to make it easier to visually compare.



(a) Distribution and convergence of the nucleosome model with a remodeller for bias to the left (negative x-direction).

(b) Distribution and convergence of the nucleosome model with a remodeller for bias to the right (positive x-direction).

Figure 15: Distribution and convergence of the nucleosome model with bias.

A Additional Material

A.1 Problem 2c

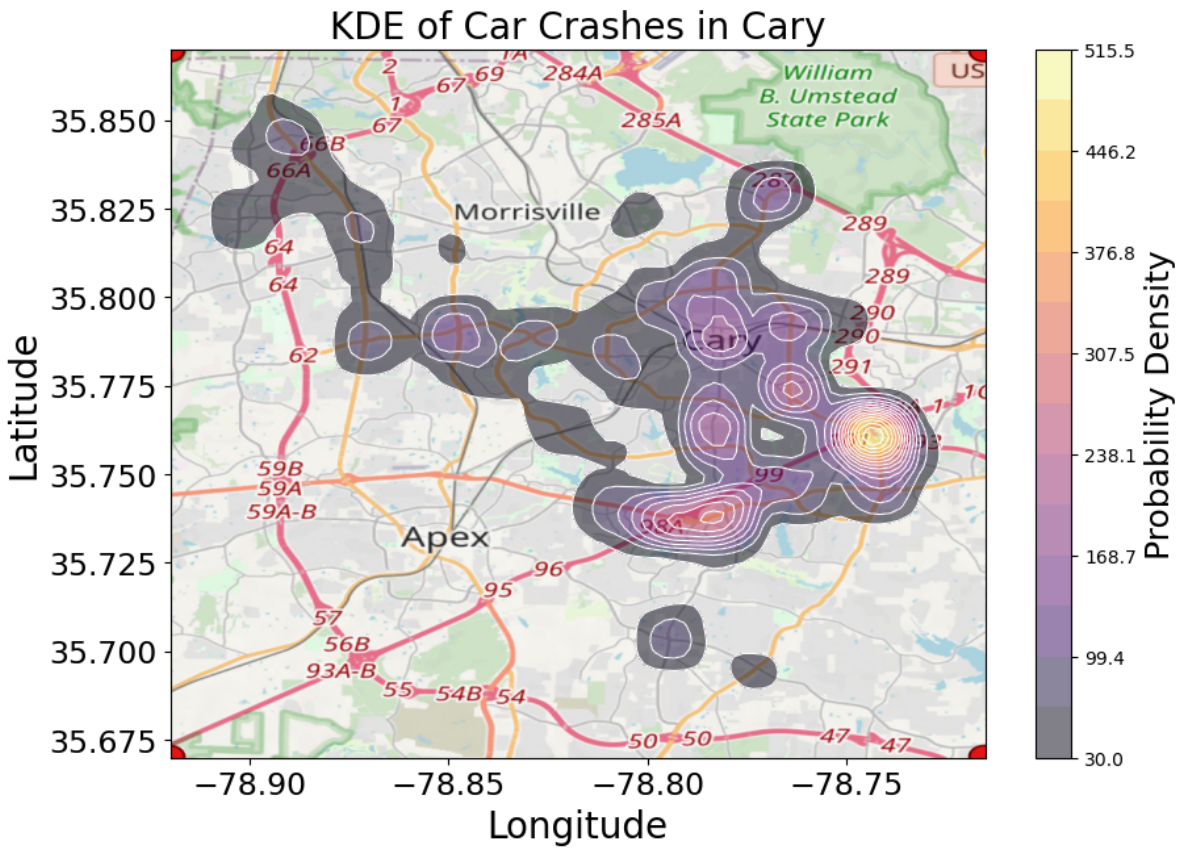


Figure 16: Estimated PDF overlaid on a screenshot from OpenStreetMap of the corresponding geographic area.